State licensing pipeline — handoff documentation

Generated 2026-07-07 from repo commit `c7715e06` by `python -m pipeline.client.handoff_doc`. Every number below was computed from the repository's actual contents when this was generated — none are hand-maintained. If this document is more than a few weeks old, re-run that command to get current numbers.

For engineers: this document is the plain-English overview. The technical entry points are `CLAUDE.md` (repo root), `pipeline/client/README.md`, and `pipeline/config/README.md`. Everything below is also inspectable in code.

1. What this is, and what you already have

This repository turns state licensing law into structured, comparable data: for each occupation and each state, what license (if any) is required, what it takes to get one, and what the license lets you do. The results are published as an interactive website.

The most important thing to know: **the raw law text is already in this repository.** Collecting statute text from the web is by far the most painful part of this work (see section 6), and we have already done it broadly — 3,090 of the 4,700 possible occupation/state combinations (94 occupations × 50 states, i.e. 66% of the grid) have their law text saved here. Day-to-day work — turning that saved text into published data — **never touches the web-scraping machinery.** You will not need to scrape anything unless you want brand-new material or a refresh of old material.

2. What's easy, what's cheap, and what's hard

The pipeline runs in stages, and they cost very different amounts of money and effort. Current state of the grid, as of generation:

	Count	What it means
Published	13 occupations	Site data built and ready to serve
Extracted, not published	49 occupations	Structured data exists for at least some states; publishing is a free command
Ready to process	2,055 occupation/state cells	Law text on disk; needs only the paid AI steps (~$0.20/cell, ~$401.70 for all of them)
Blocked on a schema	285 cells (24 occupations)	Law text on disk, but no data template designed yet (a one-time, low-cost AI step per occupation)
Needs a fetch	131 cells	Law located but text never pulled — requires the scraping setup

A fetch is running right now (checked live at generation time) — some of the 131 cells below are actively being pulled as this document was generated, so that count is falling.

The 13 published occupations: architecture, construction_trades, dentistry, dietetics, electrician, hvac, medicine, nursing, optometry, plumbing, podiatry, professional_engineering, public_accountancy.

In effort terms:

- **Free, minutes:** publish an occupation whose data is already extracted; re-run the deterministic cleanup step.
- **Cheap, no code:** run an already-fetched occupation/state through the two paid AI steps (about $0.20 per occupation/state cell). "Already fetched" means the raw statute text is saved in this repository — the AI steps just haven't been paid for yet. The dashboard (section 3) shows exactly which cells these are and quotes the cost before anything is spent.
- **Medium:** add a new occupation. Sometimes its law is already inside chapters we've pulled (free reuse); it needs a one-time "schema" (data template) designed for it, which is an AI-assisted step; and some states may need a fetch.
- **Hard, human-supervised:** fetching new or updated statute text from the web (section 6). Plan for a person to be around while it runs.

3. The tools you'll actually use

What you need: Python (the repo ships a ready environment — see `pipeline/client/README.md`), an Anthropic API key for the two paid steps, and access to the site's hosting (Vercel) for publishing. Chrome is needed only for fetching.

Three commands cover everything except fetching (run from the repo root, using the project's Python environment):

1. **Status** — `python -m pipeline.client.status` — recomputes, for every occupation and state, what stage it's at, whether it's current or stale, and what it would cost to finish.
2. **Dashboard** — `python -m pipeline.client.dashboard` — builds an offline web page (`pipeline/results/status/dashboard.html`, open it in any browser) where you tick the occupations you want, see the total cost, and get the exact commands to run.
3. **Runner** — `python -m pipeline.client.run ...` — executes your picks: `clean` (free), `trim` and `extract` (the paid steps — each prints its cost estimate and waits for a typed "yes" before spending anything), `publish` (free).

Nothing in these tools fetches from the web or spends money on its own. The plain-English walkthrough, including re-fetching, is `docs/runbooks/client_refetch.md`.

4. What it costs — and what the costs depend on

Two stages call an AI model and cost money; everything else is free:

Stage	Model used today	Est. cost
Trim — keep only the occupation's parts of the law	`claude-haiku-4-5-20251001`	~$0.05
Extract — turn trimmed text into structured data	`claude-sonnet-4-6`	~$0.15
Schema discovery — design an occupation's data template	`claude-sonnet-4-6`	small, one-time

Three things to understand about these numbers:

- **They are estimates tied to today's models and prices.** The model names above were read from the code at generation time. Anthropic's per-token prices change, and bulk runs go through Anthropic's discounted batch API. Treat the per-cell figures as planning estimates, not invoices — the runner shows the estimate and asks before spending.
- **Swapping models is easy.** Each stage's model is a single setting (a `--model` flag / one constant per stage module). Moving to a cheaper, better, or different vendor's model is a small change; the prompts themselves live in a plain-text config file (`pipeline/config/prompts.yaml`), not in code.
- **The costs are per-run, not per-view.** The website serves previously-built data; people browsing the site cost nothing.

5. Everything adjustable lives in config files, not code

The pipeline's decisions — which occupations exist, how to recognize each one in a state's law, per-state quirks, what the AI is asked, rate limits — are all plain-text YAML files in `pipeline/config/`, each documented in `pipeline/config/README.md`:

- `professions.yaml` — the occupation list and the patterns used to find each one.
- `state_structure.yaml` — per-state source and layout overrides (the single source of truth for "where does this state keep this law").
- `prompts.yaml` — the instructions given to the AI at each paid step.
- `limits.yaml` — every numeric knob (fetch delays, caps, thresholds).

A non-programmer can read these; a technical person can change them without touching Python.

6. Fetching: the hard part (and why you can usually skip it)

A fetch is running right now (checked live at generation time) — some of the 131 cells below are actively being pulled as this document was generated, so that count is falling.

Statute text comes from **Justia** (law.justia.com), a free legal-code aggregator — 2,963 of the 3,007 saved corpora were pulled from it, covering 49 of the 50 states. **Pennsylvania** is the exception: its statutes come from the state legislature's own site (www.legis.state.pa.us). (These counts are measured from the saved files' recorded URLs, not asserted.)

Fetching is not fully autonomous — plan to run it yourself, and expect it to need attention. Justia blocks automated requests with a bot-detection challenge, and the only reliable way past it is a real, visible Chrome browser window running on someone's own computer. In practice: re-fetching or fetching new statute text isn't a cloud job you kick off and forget — it runs on your machine, it can take hours for a large batch, and every so often the challenge reappears and needs a person to click through it in that browser window. We've built it to restart itself automatically after a crash or if the computer goes to sleep, so it's more resilient than a plain script, but it is not "set and forget." Budget for someone to check on it periodically while it runs.

Two related points:

- **Statute text changes roughly once a year** (an assumption, not a measurement), so refreshes are periodic, deliberate events — the pipeline never queries these sites in real time, and nothing re-fetches automatically.

- **Finding the law is often harder than fetching it.** Many occupations' licensing rules aren't a standalone chapter — they're a few sections buried inside a much broader one (one insurance-agent type inside the entire insurance code). The repo includes automated locator tooling for this, and the per-state mappings it produces are saved in config, so that work doesn't have to be redone.

7. How much to trust the data

- **It's AI-extracted, with checks.** Structured records are extracted by an AI model from statute text, then run through automated consistency checks (`pipeline/eval/check_consistency.py`). It can still be wrong; every record links back to its source law so a human can verify.
- **"Done" doesn't mean "done forever."** Each cell's output depends on its inputs (the law text) and its recipe (prompt + schema + model + code). The repo stamps every output and can tell you whether it's *current*, *stale* (an input or the recipe changed since it was built), or *unstamped* (built before tracking existed). Right now: 750 extracted cells — 350 current, 0 stale, 400 unstamped. The dashboard color-codes this.
- **Some blanks are deliberate.** A hand-verified list (`pipeline/config/verified_absences.yaml`) records occupation/state pairs where the state genuinely has no license — shown as "not state-regulated (verified)," distinct from "we haven't processed this yet."
- **Having a chapter isn't the same as having every relevant law.** Some states put discipline, fees, or hearing rules in shared "umbrella" chapters that apply to many occupations; the repo detects and pulls these, but completeness auditing is ongoing.

Regenerate with `python -m pipeline.client.handoff_doc [--pdf]`.